

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)

Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа №3
по дисциплине: ТАиФЯ
по теме: Регулярные языки и конечные распознаватели

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: Рязанов Ю. Д.

Белгород 2024

Цель работы: изучить основные способы задания регулярных языков, способы построения, алгоритмы преобразования, анализа и реализации конечных распознавателей.

З а д а н и е

1. Построить минимальный детерминированный конечный распознаватель заданного языка (см. варианты заданий).

2. Написать программу-распознаватель компиляционного и интерпретационного типа.

Исходные данные: строка.

Результат: “допустить” — если строка представляет собой цепочку заданного языка;

“отвергнуть” — в противном случае.

3. Написать программу, которая оставляет в исходном текстовом файле только те строки, которые представляют собой цепочки заданного языка.

4. Написать программу, которая исключает из исходного текстового файла строки, являющиеся цепочками заданного языка.

Вариант 9

Язык слов, правильно разбитых на две части.

Части слова разделяются символом - . Слово считается правильно разбитым на две части, если: — первая часть заканчивается последовательностью состоящей из согласной и гласной буквы, а вторая часть начинается гласной, за которой идёт хотя бы одна буква (буква й при этом рассматривается вместе с предшествующей гласной как единое целое);

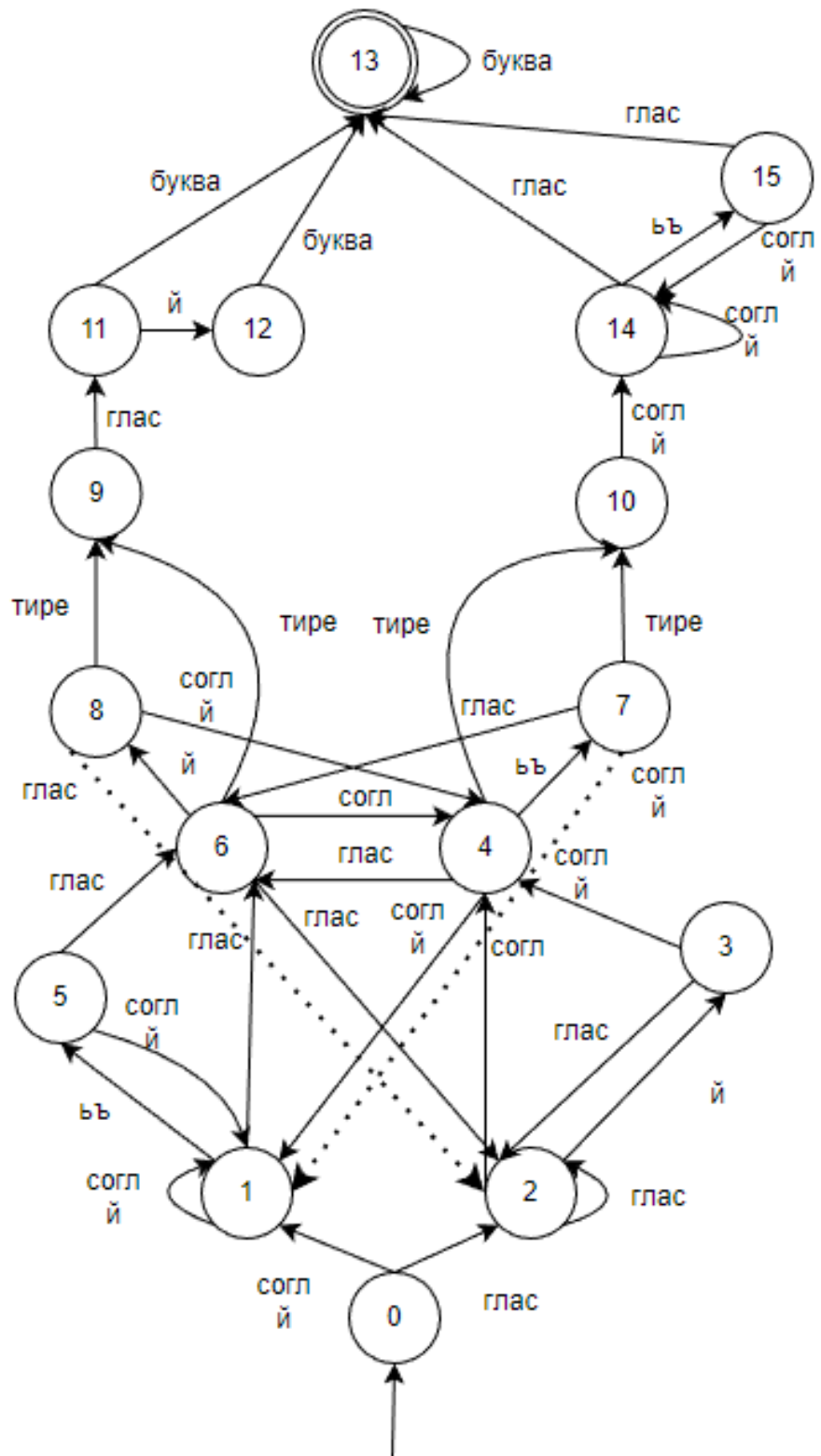
— первая часть заканчивается последовательностью, состоящей из гласной и согласной буквы, а вторая часть начинается согласной и в оставшейся части имеется хотя бы одна гласная (буквы ь и ъ вместе с предшествующей согласной рассматриваются как единое целое).

Выполнение работы:

Задание №1:

Введем следующие обозначения:

- согл – согласные (без «й»)
- глас – гласные
- буква – любая буква: гласная, согласная, твердый/мягкий знак
- тире – символ «-»



Данный распознаватель является детерминированным.

Отметим, что графическое представление вышло довольно сложным, т.к. условия задачи варианта приводят к тому, что мы имеем несколько вариантов путей во многих случаях.

Например, т.к. «й» рассматривается с гласной как единое целое – то везде, куда можно попасть из гласной – можно попасть также из комбинации гласная + «й», при этом необходимо учитывать, что следующая «й» уже будет считаться согласной сама по себе и т.п.

Две длинные стрелки внизу схемы сделаны пунктиром для улучшения видимости.

Табличный вид

														1		
	S 0	S 1	S 2	S 3	S4	S 5	S 6	S7	S 8	S9	S1 0	S11	S1 2	S1 3	S1 4	S1 5
глас	S 2	S 6	S 2	S 2	S6	S 6	S 2	S6	S 2	S1 1					S1 3	S1 3
согл	S 1	S 1	S 4	S 4	S1	S 1	S 4	S1	S 4		S1 4				S1 4	S1 4
й	S 1	S 1	S 3	S 4	S1	S 1	S 8	S1	S 4		S1 4	S1 2			S1 4	S1 4
ьъ		S 5			S7										S1 5	
букв а												S1 3	S1 3	S1 3		
тире					S1 0		S 9	S1 0	S 9							

Произведем минимизацию.

Для начала определим, что все вершины достижимы – произведем обход в глубину графа состояний: S0, S1, S5, S6, S8, S9, S11, S13, S12, S4, S10, S14, S15, S7, S2, S3

Все вершины достижимы.

Произведем поиск и замену эквивалентных между собой состояний одним состоянием.

Воспользуемся нахождением классов эквивалентных состояний, которые станут состояниями нашего минимального распознавателя.

Последний столбец справа соответствует состоянию ошибки.

														1			
	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	
глас	S2	S6	S2	S2	S6	S6	S2	S6	S2	S11					S13	S13	
согл	S1	S1	S4	S4	S1	S1	S4	S1	S4		S14				S14	S14	
й	S1	S1	S3	S4	S1	S1	S8	S1	S4		S14	S12			S14	S14	
ьъ		S5			S7										S15		
букв а												S13	S13	S13			
тире					S10		S9	S10	S9								

Отвергающие состояния:

{S0, S1, S2, S3, S4, S5, S6, S7, S8, S9, S10, S11, S12, S14, S15}

Объединим их в класс K1, а допускающие {s13} в класс K2

	K1													K2	K1		
	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	
глас	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K2	K2	K1
согл	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1
й	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1
ьъ	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1
буква	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K2	K2	K2	K1	K1	K1
тире	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1

	K1											K3		K2	K4		K1
	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	
глас	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K2	K2	K1
согл	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K4	K4	K1
й	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K3	K1	K1	K4	K4	K1
ьъ	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K4	K1	K1
буква	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K2	K2	K2	K1	K1	K1
тире	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1

	K1											K6	K3	K2	K5	K4	K1
	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	
глас	K1	K1	K1	K1	K1	K1	K1	K1	K1	K6	K1	K1	K1	K1	K2	K2	K1
согл	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K5	K1	K1	K1	K5	K5	K1
й	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K5	K3	K1	K1	K5	K5	K1
ьъ	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K4	K1	K1
буква	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K2	K2	K2	K1	K1	K1
тире	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1

	K1									K8	K7	K6	K3	K2	K5	K4	K1
	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	
глас	K1	K1	K1	K1	K1	K1	K1	K1	K1	K6	K1	K1	K1	K1	K2	K2	K1
согл	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K5	K1	K1	K1	K5	K5	K1
й	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K5	K3	K1	K1	K5	K5	K1
ьъ	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K4	K1	K1
буква	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K2	K2	K2	K1	K1	K1
тире	K1	K1	K1	K1	K7	K1	K8	K7	K8	K1	K1	K1	K1	K1	K1	K1	K1

	K1				K10	K1	K9	K10	K9	K8	K7	K6	K3	K2	K5	K4	K1
	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	
глас	K1	K19	K1	K1	K9	K9	K1	K9	K1	K6	K1	K1	K1	K1	K2	K2	K1
согл	K1	K1	K10	K10	K1	K1	K10	K1	K10	K1	K5	K1	K1	K1	K5	K5	K1
й	K1	K1	K1	K10	K1	K1	K9	K1	K10	K1	K5	K3	K1	K1	K5	K5	K1
ьъ	K1	K1	K1	K1	K10	K1	K1	K1	K1	K1	K1	K1	K1	K1	K4	K1	K1
буква	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K2	K2	K2	K1	K1	K1
тире	K1	K1	K1	K1	K7	K1	K8	K7	K8	K1	K1	K1	K1	K1	K1	K1	K1

	K17	K16	K15	K14	K12	K13	K11	K10	K9	K8	K7	K6	K3	K2	K5	K4	K1
	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	
глас	K15	K11	K15	K15	K11	K11	K15	K11	K15	K6	K1	K1	K1	K1	K2	K2	K1
согл	K16	K16	K12	K12	K16	K16	K12	K16	K12	K1	K5	K1	K1	K1	K5	K5	K1
й	K16	K16	K14	K12	K16	K16	K9	K16	K12	K1	K5	K3	K1	K1	K5	K5	K1
ьъ	K1	K13	K1	K1	K10	K1	K1	K1	K1	K1	K1	K1	K1	K1	K4	K1	K1
буква	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K1	K2	K2	K2	K1	K1	K1
тире	K1	K1	K1	K1	K7	K1	K8	K7	K8	K1	K1	K1	K1	K1	K1	K1	K1

Изначальный распознаватель был минимальным.

Задание №2:

Программа:

```
def is_consonant(c : str) -> bool:
    return c.lower() in 'бвгджзклмнпрстфхцчшщ'

def is_vowel(c : str) -> bool:
    return c.lower() in 'аяуюеёёэиы'

def is_consonant_full(c : str) -> bool:
    return c.lower() in 'бвгджзклмнпрстфхцчшщй'

def is_sym(c : str) -> bool:
    return c.lower() in 'бвгджзклмнпрстфхцчшщйаяуюеёёэиы'

def report_error(state_num : int, c : str) -> bool:
    print('{0}Error. S = {1}, character = <{2}>, {3}'.format("\033[31m",
state_num, c, "\033[0m"))
    return True, c

def is_sh(c : str):
    return c.lower() in 'ьъ'

class ParseResult:
    def __init__(self, is_right, state, character) -> None:
        self.is_right = is_right
        self.state = state
        self.character = character

def is splitted_right(line : str) -> ParseResult:
    line = line.strip()
    S = 0
    err = True
    char = ''
    for c in line:
        match S:
            case 0:
                if is_consonant_full(c):
                    S = 1
                elif is_vowel(c):
                    S = 2
                else:
                    err, char = report_error(S, c)
                    break
            case 1:
                if is_consonant_full(c):
                    S = 1
                elif is_sh(c):
                    S = 5
                elif is_vowel(c):
                    S = 6
                else:
                    err, char = report_error(S, c)
                    break
            case 2:
                if is_vowel(c):
                    S = 2
                elif c == 'й':
                    S = 3
                elif is_consonant(c):
                    S = 4
                else:
                    err, char = report_error(S, c)
```



```

        break
    case 3:
        if is_vowel(c):
            S = 2
        elif is_consonant_full(c):
            S = 4
        else:
            err, char = report_error(S, c)
            break
    case 4:
        if is_vowel(c):
            S = 6
        elif is_sh(c):
            S = 7
        elif c == '-':
            S = 10
        elif is_consonant_full(c):
            S = 1
        else:
            err, char = report_error(S, c)
            break
    case 5:
        if is_consonant_full(c):
            S = 1
        elif is_vowel(c):
            S = 6
        else:
            err, char = report_error(S, c)
            break
    case 6:
        if is_consonant(c):
            S = 4
        elif is_vowel(c):
            S = 2
        elif c == 'й':
            S = 8
        elif c == '-':
            S = 9
        else:
            err, char = report_error(S, c)
            break
    case 7:
        if c == '-':
            S = 10
        elif is_vowel(c):
            S = 6
        elif is_consonant_full(c):
            S = 1
        else:
            err, char = report_error(S, c)
            break
    case 8:
        if is_consonant_full(c):
            S = 4
        elif is_vowel(c):
            S = 2
        elif c == '-':
            S = 9
        else:
            err, char = report_error(S, c)
            break
    case 9:
        if is_vowel(c):
            S = 11

```

```

        else:
            err, char = report_error(S, c)
            break
    case 10:
        if is_consonant_full(c):
            S = 14
        else:
            err, char = report_error(S, c)
            break
    case 11:
        if c == 'й':
            S = 12
        elif is_sym(c):
            S = 13
        else:
            err, char = report_error(S, c)
            break
    case 12:
        if is_sym(c):
            S = 13
        else:
            err, char = report_error(S, c)
            break
    case 13:
        if is_sym(c):
            S = 13
            err = False
        else:
            err, char = report_error(S, c)
            break
    case 14:
        if is_consonant_full(c):
            S = 14
        elif is_sh(c):
            S = 15
        elif is_vowel(c):
            S = 13
        else:
            err, char = report_error(S, c)
            break
    case 15:
        if is_vowel(c):
            S = 13
        elif is_consonant_full(c):
            S = 14
        else:
            err, char = report_error(S, c)
            break
    return ParseResult(not err, S, char)

def main():
    word = 'прыйй-евет'
    print('Разбираемое слово <{0}>'.format(word))
    res = is_splitting_right(word)
    if res.is_right:
        print("Слово разбито верно")
    else:
        print("Слово разбито неверно")
    if not res.is_right:
        print("Распознаватель был в состоянии S{0}, символ, вызвавший ошибку <{1}>.".format(res.state, res.character))

```

```
if __name__ == "__main__":  
    main()
```

Результаты работы:

```
Разбираемое слово <прый-евет>  
Слово разбито верно
```

```
Разбираемое слово <прѝи-евет>  
Слово разбито верно
```

```
Разбираемое слово <при-евет>  
Слово разбито верно
```

```
Разбираемое слово <при-вет>  
Error. S = 9, character = <в>,  
Слово разбито неверно  
Распознаватель был в состоянии S9, символ, вызвавший ошибку <в>.
```

```
Разбираемое слово <пар-вет>  
Слово разбито верно
```

Т.к. обращений к несуществующим переходам не было – то символа, вызвавшего ошибку, нет – то слово все равно разбито неверно, т.к. конечное состояние не является допускающим.

```
Разбираемое слово <парвет>  
Слово разбито неверно  
Распознаватель был в состоянии S4, символ, вызвавший ошибку <>.
```

```
Разбираемое слово <парrrrrrr-вет>  
Error. S = 1, character = <->,  
Слово разбито неверно  
Распознаватель был в состоянии S1, символ, вызвавший ошибку <->.
```

```
Разбираемое слово <парrrrrrrъез-вет>  
Слово разбито верно
```

Программа:

```
def is_consonant(c : str) -> bool:
    return c.lower() in 'бвгджзклмнпрстфхцчщщ'

def is_vowel(c : str) -> bool:
    return c.lower() in 'аяуюеёёэиы'

def is_consonant_full(c : str) -> bool:
    return c.lower() in 'бвгджзклмнпрстфхцчщщй'

def is_sym(c : str) -> bool:
    return c.lower() in 'бвгджзклмнпрстфхцчщщйаяуюеёёэиы'

def report_error(state_num : int, c : str) -> bool:
    print('{0}Error. S = {1}, character = <{2}>, {3}'.format("\033[31m",
state_num, c, "\033[0m"))
    return True, c

def is_sh(c : str) -> bool:
    return c.lower() in 'ьъ'

def get_index(c : str) -> int:
    if is_vowel(c):
        return 0
    elif is_consonant(c):
        return 1
    elif c == 'й':
        return 2
    elif is_sh(c):
        return 3
    elif c == '-':
        return 4
    else:
        raise(Exception("Wrong index"))

class ParseResult:
    def __init__(self, is_right, state, character) -> None:
        self.is_right = is_right
        self.state = state
        self.character = character

class Parser:
    def __init__(self, fileName) -> None:
        self.matr = []
        self.S = 0
        self.status = ParseResult(False, -100, '')
        with open(fileName, "r") as f:
            lines = f.readlines()
            for i in range(len(lines)):
                self.matr.append([])
                line = lines[i].strip()
                states = line.split(' ')
                for state in states:
                    self.matr[i].append(int(state) if state != '.' else -100)

    def parse(self, str) -> ParseResult:
        for c in str:
            new_state = self.matr[get_index(c)][self.S]
            self.status.state = self.S
            self.status.character = c
            if new_state != -100:
                self.S = new_state
            if new_state == 13:
```

```

        self.status.is_right = True
    else:
        report_error(self.S, c)
        self.status.is_right = False
        return
    self.status.character = ''

def main():
    word = 'прыйй-евет'
    print('Разбираемое слово <{0}>'.format(word))
    parser = Parser("table.txt")
    parser.parse(word)
    res = parser.status
    if res.is_right:
        print("Слово разбито верно")
    else:
        print("Слово разбито неверно")
    if not res.is_right:
        print("Распознаватель был в состоянии S{0}, символ, вызвавший ошибку <{1}>.".format(res.state, res.character))

if __name__ == "__main__":
    main()

```

Таблица переходов, немного модифицированная под интерпретационный вариант:

1	2	6	2	2	6	6	2	6	2	11	.	13	13	13	13	13
2	1	1	4	4	1	1	4	1	4	.	14	13	13	13	14	14
3	1	1	3	4	1	1	8	1	4	.	14	12	13	13	14	14
4	.	5	.	.	7	.	.	.	.	.	.	13	13	13	15	.
5	.	.	.	.	10	.	9	10	9	.	.	.	.	.	.	.

Результаты работы:

Разбираемое слово <паррррррьез-вет>
Слово разбито верно

Разбираемое слово <паррррррр-вет>

Error. S = 1, character = <->,

Слово разбито неверно

Распознаватель был в состоянии S1, символ, вызвавший ошибку <->.

Разбираемое слово <парррррррвет>

Слово разбито неверно

Распознаватель был в состоянии S6, символ, вызвавший ошибку <>.

Разбираемое слово <пар-вт>

Слово разбито неверно

Распознаватель был в состоянии S14, символ, вызвавший ошибку <>.

Разбираемое слово <пар-вет>

Слово разбито верно

Разбираемое слово <прый-евет>

Слово разбито верно

Задание №3:

```
from main import isSplittedRight

def main():
    fileName = input("Введите имя файла: ")
    good = []
    lines = []
    with open(fileName, "r") as f:
        lines = f.readlines()
        for i in range(len(lines)):
            if isSplittedRight(lines[i]).is_right:
                good.append(i)
    with open(fileName, "w") as f:
        for i in good:
            f.write(lines[i])

if __name__ == "__main__":
    main()
```

Результаты работы:

test.txt

1	при-вет
2	привет
3	при-евет
4	прий-евет
5	перь-вет
6	перь-вт

test.txt

1	при-евет
2	прий-евет
3	перь-вет

test.txt

1	при-евет
2	прий-евет
3	перь-вет

test.txt

1	при-евет
2	прий-евет
3	перь-вет

test.txt

1	при-вет
2	привет
3	перь-вт

test.txt

1	
---	--

Задание №4:

```
from main import isSplittedRight

def main():
    fileName = input("Введите имя файла: ")
    good = []
    lines = []
    with open(fileName, "r") as f:
        lines = f.readlines()
        for i in range(len(lines)):
            if not isSplittedRight(lines[i]).is_right:
                good.append(i)
    with open(fileName, "w") as f:
        for i in good:
            f.write(lines[i])

if __name__ == "__main__":
    main()
```

Результаты работы:

test.txt

1	при-вет
2	привет
3	при-евет
4	прий-евет
5	перь-вет
6	перь-вт

1	при-вет
2	привет
3	перь-вт

1	при-вет
2	привет
3	перь-вт

1	при-вет
2	привет
3	перь-вт

1	при-евет
2	прий-евет
3	перь-вет

1	
---	--

Вывод:

Мы изучили основные способы задания регулярных языков, способы построения, алгоритмы преобразования, анализа и реализации конечных распознавателей.